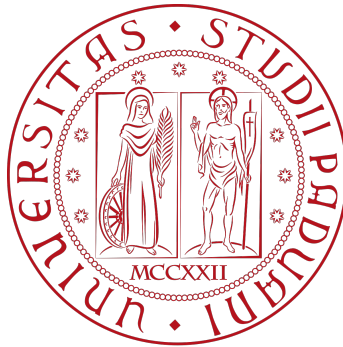




UNIVERSITÀ DEGLI STUDI DI PADOVA

Scuola di Scienze
Corso di Laurea in Informatica

Progetto - Basi di Dati

Escursioni

Studenti:

**Giovanni Veronese
Sebastiano Lorenzi**

Docente:

Prof. Massimiliano de Leoni

Anno Accademico: 2025-2026

Indice

1	Abstract	1
2	Analisi dei requisiti	1
3	Progettazione concettuale	3
4	Progettazione logica	4
4.1	Analisi delle ridondanze	4
4.1.1	Con ridondanza	4
4.1.2	Senza ridondanza	5
4.1.3	Conclusioni	5
4.2	Eliminazione delle generalizzazioni	5
4.2.1	Generalizzazione Persona	6
4.2.2	Generalizzazione Tappa	6
4.3	Partizionamento e accorpamento di entità e relazioni	6
4.4	Scelta degli identificatori primari	6
4.4.1	Persona	6
4.4.2	Guida ed Escursionista	6
4.4.3	Parco	7
4.4.4	Sentiero	7
4.4.5	Escursione	7
4.4.6	Tappa	7
4.4.7	Rifugio e Cima	7
4.5	Schema Relazionale	7
5	Implementazione in PostgreSQL e definizione delle Query	9
5.1	Definizione delle Query	9
5.2	Creazione degli indici	13

1 Abstract

Questo progetto sviluppa una base di dati per gestire la ricerca e la partecipazione a escursioni in montagna. Il principale obiettivo è poter ricercare l'itinerario su misura per sé, partendo da criteri molto diversi: se si preferisce l'aiuto di un esperto si può vedere in quali date sono previste le escursioni guidate. È pensato anche per essere un sistema che incrocia sentieri e punti d'interesse, perciò si può partire con dei luoghi in cui arrivare e trovare il percorso o al contrario dal percorso scoprire le tappe che si attraversano. I sentieri appartengono a un parco naturale che sulla sua superficie può ospitare diversi sentieri e ancor più tappe. Un sentiero ha un luogo di partenza e uno di arrivo, che possono coincidere. Uno dei punti d'interesse può essere un rifugio, affidato a un responsabile e con possibilità di avere un numero di posti letto per il pernottamento. Le persone iscritte alla lista sono identificate dal codice fiscale, possono essere normali escursionisti o delle guide, che organizzano delle escursioni guidate in una determinata data a cui possono partecipare un certo numero di persone.

2 Analisi dei requisiti

Seguono i requisiti della base di dati:

Persona. l'essere inserita nel sistema è già una conferma di aver superato i requisiti fisici e medici. Ogni persona è identificata da

- **codice fiscale** che distingue le persone in modo univoco
- **nome**
- **cognome**
- **tipo persona** ovvero se è un escursionista o una guida

Ogni persona può partecipare a un evento organizzato come **escursionista** o come **guida**.

Guida. eredita tutti gli attributi di Persona, e aggiunge un attributo

- **numero tesserino**, ogni guida deve essere identificata come guida abilitata tramite un codice

Escursionista. eredita tutti gli attributi di Persona, ma implementa l'attributo

- **scadenza certificato medico** che non permette la partecipazione a una Escursione se la data di scadenza non è successiva

Tappa. è identificata da un codice univoco e permette di sapere i luoghi dove passa un Sentiero con i seguenti attributi:

- **id tappa** per non creare ambiguità,
- **nome** spiega o descrive il tipo di luogo
- **altitudine** in metri, trattandosi di montagna è un dato utile da avere
- **provincia** identifica la provincia in cui si trova quel luogo
- **copertura di rete** un valore binario che informa la presenza (1) o meno (0) di segnale telefonico. In caso di dubbio il dato è 0.

Rifugio. eredita gli attributi di Tappa, e fornisce informazioni aggiuntive essendo una Tappa strategica

- **id tappa** ereditata da Tappa
- **posti letto** indica la presenza e la possibilità di pernottamento

- **codice fiscale del gestore**, un rifugio inserito nell'elenco deve necessariamente avere una persona di riferimento, iscritta all'elenco Persona
- **proprietà** i rifugi possono essere proprietà di enti locali, privati o delle varie sezioni del Club Alpino Italiano

Cima. eredita gli attributi di Tappa, e fornisce una informazione in più

- **id tappa** ereditata da Tappa
- **gruppo montuoso** indica l'appartenenza della specifica cima all'insieme di montagne dove si colloca

Parco. indica l'area geografica dove sarà collocato il Sentiero

- **nome** è la descrizione testuale del parco Naturale o Regionale
- **regione** descrive la regione di appartenenza del parco
- **superficie** l'area espressa in km² del territorio descritto

Sentiero. un percorso che ha un inizio e una fine che possono coincidere, descritto dai seguenti attributi

- **codice sentiero** codice CAI assegnato al sentiero, univoco e utile per le indicazioni
- **nome** descrizione testuale del percorso
- **difficoltà** indica all'escursionista la difficoltà stimata del percorso
- **lunghezza** espressa in km, indica quanto è lungo il percorso
- **tempo di percorrenza** espressa in minuti, suggerisce il tempo standard di percorrenza
- **dislivello** espresso in metri, indica il dislivello positivo. Insieme alla lunghezza permette di farsi un'idea precisa dell'impegno necessario per una particolare escursione
- **numero di tappe** un valore intero che indica le tappe salvate nel database che saranno attraversate in questo percorso, senza contare il punto di partenza e il punto di arrivo
- **nome del parco** indica in quale parco Nazionale o Regionale appartiene il percorso. Molti sentieri non appartengono a uno specifico parco ma al generico territorio regionale
- **regione del parco** regione di appartenenza del percorso
- **id tappa di partenza** indica il luogo da cui parte il percorso
- **id tappa di arrivo** indica il luogo in cui il percorso finisce. Può corrispondere alla partenza in caso di percorsi ad anello

Escursione. è un evento a cui ci si può iscrivere e partecipare, caratterizzato dalle seguenti informazioni

- **id escursione** univoco per ogni escursione programmata
- **data programmata** stabilisce il giorno per cui è previsto l'evento
- **costo** la quota necessaria per partecipare
- **codice sentiero** indica quale sentiero si andrà a percorrere
- **posti disponibili** indica la disponibilità dei posti per quello specifico evento. Se la difficoltà del sentiero è maggiore, i posti disponibili sono tendenzialmente più bassi per una maggiore sicurezza

- **cf guida** identifica la persona che sarà alla guida del gruppo.

La tabella **attraversa** tiene traccia di tutte le tappe collegate ai sentieri, collegando

- **codice del sentiero**
- **id tappa**

La tabella **partecipa** crea un collegamento tra gli escursionisti e la partecipazione a ogni escursione, creando uno storico

- **cf escursionista**
- **id escursione**

3 Progettazione concettuale

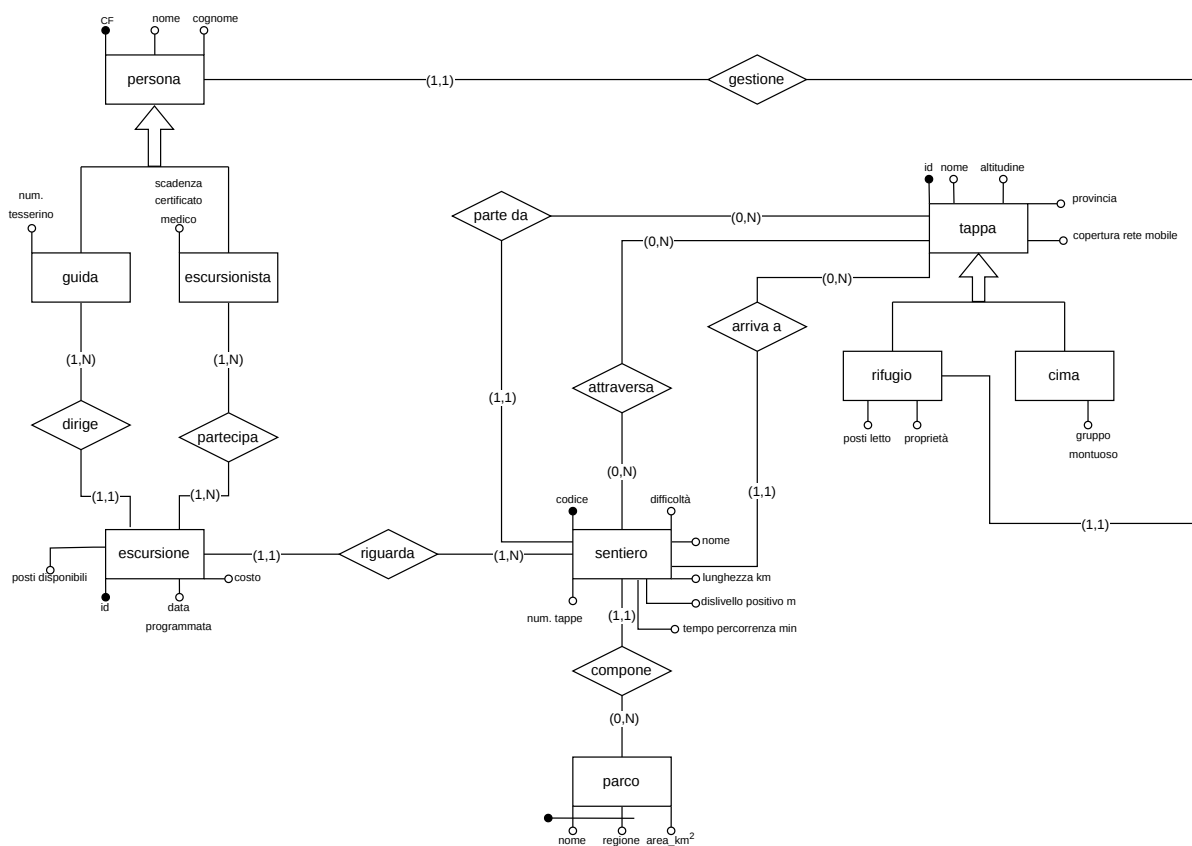


Figura 1: Diagramma E/R relativo alle escursioni.

La Figura 1 mostra i requisiti descritti nella sezione 2.

Nel database vengono rappresentate le entità principali delle escursioni: Persona, Guida, Escursionista, Tappa, Rifugio, Cima, Parco, Sentiero ed Escursione. Le relazioni tra questi concetti consentono scegliere un sentiero, anche in base alle tappe che lo compongono; le escursioni organizzate da una guida e la partecipazione degli escursionisti. La specializzazione tra Persona, Guida ed Escursionista consente di distinguere in modo chiaro i ruoli del senza duplicare informazioni. Allo stesso modo funzionano le specializzazioni di Tappa denominate Cima e Rifugio. Inoltre, la relazione tra Sentiero e Parco permette di associare ogni itinerario alla zona geografica di riferimento, mentre quella tra Rifugio e Persona definisce l'unico responsabile del luogo. Il sistema è progettato sia per gestire i dati statici sia i dati relativi all'organizzazione delle escursioni oppure fare un semplice piano di ricerca a scopo informativo.

4 Progettazione logica

4.1 Analisi delle ridondanze

L'attributo **num_tappe** dell'entità **Sentiero**, che memorizza il numero di tappe appartenenti a un sentiero, rappresenta una ridondanza. Tale informazione può infatti essere ricavata contando il numero di tappe associate al sentiero tramite la relazione **attraversa**.

Le operazioni considerate nell'analisi sono:

- **Operazione 1 (10 volte):** inserimento di una nuova tappa in un sentiero.
- **Operazione 2 (100 volte):** visualizzazione del numero di tappe di un sentiero.

Assumendo i seguenti volumi nella base di dati:

Concetto	Costrutto	Volume
SENTIERO	E	100
TAPPA	E	400
ATTRaversa	R	1000

Tabella 1: Volumi considerati per l'analisi delle ridondanze.

Essendo **attraversa** una relazione multi-a-multi, è possibile che diverse tappe siano condivise da più sentieri. Per questo motivo il numero di istanze della relazione è superiore al numero delle tappe.

Si assume inoltre che il costo di una scrittura sia pari a **1**, mentre quello di una lettura sia pari a **2**.

4.1.1 Con ridondanza

Operazione 1

Concetto	Costrutto	Accessi	Tipo
TAPPA	E	1	S
ATTRaversa	R	1	S
SENTIERO	E	1	L
SENTIERO	E	1	S

Tabella 2: Accessi dell'operazione 1.

Operazione eseguita **10 volte**.

Operazione 2

Concetto	Costrutto	Accessi	Tipo
SENTIERO	E	1	L

Tabella 3: Accessi dell'operazione 2.

Operazione eseguita **100 volte**.

Costo totale

$$10 \times (3 \times 2 + 1) + 100 = 170$$

Concetto	Costrutto	Accessi	Tipo
TAPPA	E	1	S
ATTRAVERSA	R	1	S

Tabella 4: Accessi dell'operazione 1.

4.1.2 Senza ridondanza

Operazione 1

Operazione eseguita **10 volte**.

Operazione 2

Poiché il numero di tappe non è memorizzato, è necessario contare le istanze della relazione **Attraversa** associate al sentiero.

Considerando:

$$1000/100 = 10$$

in media ogni sentiero attraversa **10 tappe**.

Concetto	Costrutto	Accessi	Tipo
SENTIERO	E	1	L
ATTRAVERSA	R	10	L

Tabella 5: Accessi dell'operazione 2.

Operazione eseguita **100 volte**.

Costo totale

$$10 \times (2 \times 2) + 100 \times (10 + 1) = 40 + 1100 = 1140$$

4.1.3 Conclusioni

L'analisi mostra che mantenere l'attributo ridondante **num_tappe** comporta un costo maggiore nelle operazioni di aggiornamento, ma riduce notevolmente il costo delle operazioni di consultazione, che risultano molto più frequenti. Per questo motivo è conveniente mantenere l'attributo ridondante nello schema.

4.2 Eliminazione delle generalizzazioni

Nello schema E-R sono presenti due generalizzazioni, entrambe di tipo **parziale e disgiunta**:

- **Persona**, specializzata nelle entità **Guida** ed **Escursionista**;
- **Tappa**, specializzata nelle entità **Rifugio** e **Cima**.

Per entrambe le generalizzazioni è stata adottata la strategia di mantenere la superclasse e le relative sottoclassi.

4.2.1 Generalizzazione Persona

La generalizzazione dell'entità **Persona** è stata mantenuta poiché le sottoclassi **Guida** ed **Escursionista** possiedono attributi specifici (**NumeroTesserino** e **ScadenzaIscrizione**) e partecipano a relazioni differenti con l'entità **Escursione** (**Dirige** e **Partecipa**).

L'eliminazione della superclasse avrebbe comportato la duplicazione degli attributi comuni (**CF**, **Nome** e **Cognome**) nelle due sottoclassi, introducendo una ridondanza non necessaria.

4.2.2 Generalizzazione Tappa

Anche la generalizzazione dell'entità **Tappa** è stata mantenuta.

Le sottoclassi **Rifugio** e **Cima** possiedono attributi specifici (**posti_letto** e **gruppo_montuoso**) che non sono condivisi. Inoltre, essendo la generalizzazione **parziale**, una tappa può anche non appartenere ad alcuna delle due sottoclassi, rappresentando ad esempio un punto panoramico, un bivio o un'altra tappa significativa del percorso.

Il mantenimento della superclasse consente quindi di rappresentare correttamente tutte le tipologie di tappa, evitando sia la duplicazione degli attributi comuni sia la presenza di valori nulli negli attributi specifici delle sottoclassi.

In entrambe le generalizzazioni, le sottoclassi ereditano l'identificatore della rispettiva superclasse.

4.3 Partizionamento e accorpamento di entità e relazioni

Durante la ristrutturazione dello schema E-R è stata valutata la possibilità di accorpare alcune entità e relazioni.

Non sono stati effettuati ulteriori accorpamenti, poiché le entità presenti rappresentano concetti distinti del dominio applicativo e possiedono attributi o relazioni specifiche che ne giustificano il mantenimento.

Anche le relazioni **attraversa**, **parte_da** e **arriva_a** sono state mantenute separate, in quanto descrivono vincoli semantici differenti. In particolare, **parte_da** individua la tappa iniziale di un sentiero, **arriva_a** la tappa finale, mentre **attraversa** rappresenta l'insieme delle tappe appartenenti al percorso. Un loro accorpamento avrebbe richiesto l'introduzione di attributi discriminanti e avrebbe reso il modello meno chiaro e più complesso da gestire.

Si è pertanto scelto di mantenere inalterata la struttura delle entità e delle relazioni, preservando la chiarezza dello schema concettuale e la corretta rappresentazione del dominio.

4.4 Scelta degli identificatori primari

La scelta degli identificatori primari è stata effettuata privilegiando, ove possibile, attributi naturali univoci e, negli altri casi, identificatori artificiali che garantiscono l'univocità delle istanze.

4.4.1 Persona

L'entità **Persona** è identificata dal **Codice Fiscale (cf)**, che rappresenta un identificatore naturale univoco e stabile nel tempo.

4.4.2 Guida ed Escursionista

Le entità **Guida** ed **Escursionista** ereditano l'identificatore della superclasse **Persona**. Di conseguenza, il **cf** identifica univocamente anche le istanze delle due sottoclassi.

4.4.3 Parco

L'entità **Parco** è identificata dalla coppia (**nome**, **regione**). La scelta di una chiave composta consente di distinguere univocamente i parchi, evitando possibili ambiguità nel caso di parchi con lo stesso nome situati in regioni diverse.

4.4.4 Sentiero

L'entità **Sentiero** è identificata da un **codice** univoco assegnato a ciascun percorso, evitando possibili ambiguità dovute alla presenza di sentieri con lo stesso nome.

4.4.5 Escursione

L'entità **Escursione** è identificata da un codice **id** univoco. L'utilizzo di un identificatore evita la definizione di chiavi composte e consente di distinguere univocamente più escursioni organizzate sullo stesso sentiero.

4.4.6 Tappa

L'entità **Tappa** è identificata da un **ID** univoco. L'utilizzo di un identificatore artificiale evita possibili duplicazioni dovute a tappe con lo stesso nome o situate alla medesima altitudine.

4.4.7 Rifugio e Cima

Le entità **Rifugio** e **Cima** ereditano l'identificatore dell'entità **Tappa**, garantendo l'univocità delle rispettive istanze senza introdurre ulteriori identificatori.

4.5 Schema Relazionale

Lo schema ristrutturato in Figura 2 contiene costrutti in corrispondenza dello schema relazionale. Lo schema logico è rappresentato a seguire, dove l'asterisco dopo il nome degli attributi indica quelli che ammettono valori nulli.

- **Persona**(cf, nome, cognome)
- **Guida**(cf, num_tesserino)
 - Guida.Persona → Persona.cf
- **Escursionista**(cf, scadenza_certificato_medico)
 - Escursionista.Persona → Persona.cf
- **Parco**(nome, regione, superficie_km*²)
- **Sentiero**(codice, nome, difficoltà, lunghezza, tempo_percorrenza, dislivello, num_tappe, nome_parco, regione_parco, partenza, arrivo)
 - Sentiero.(parco_nome, parco_region) → Parco.(nome, regione)
 - Sentiero.partenza → Tappa.id
 - Sentiero.arrivo → Tappa.id
- **Tappa**(id, nome, altitudine_m*, provincia*, copertura_rete_mobile)
- **Rifugio**(id_tappa, posti letto, gestore, proprietà*)
 - Rifugio.Tappa → Tappa.id

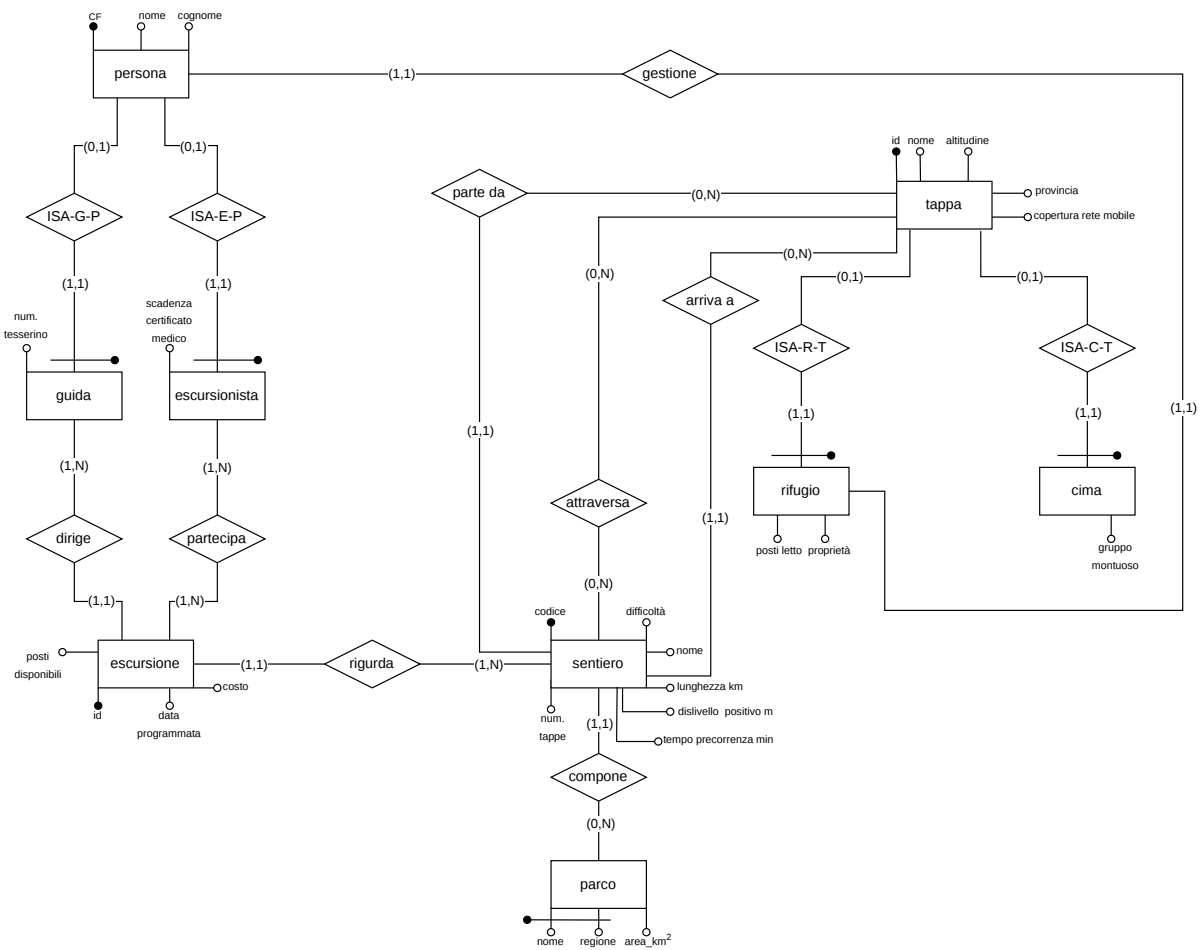


Figura 2: Diagramma E/R ristrutturato

- Rifugio.gestore → Persona.cf
- **Cima**(id_tappa, gruppo_montuoso)
 - Cima.Tappa → Tappa.ID
- **Escursione**(id, data_programmata, costo, sentiero, guida)
 - Escursione.Sentiero → Sentiero.codice
 - Escursione.Guida → Guida.Persona
- **attraversa**(Sentiero, Tappa)
 - Attraversa.Sentiero → Sentiero.codice
 - Attraversa.Tappa → Tappa.id
- **partecipa**(Escursionista, Escursione)
 - Partecipa.Escursionista → Escursionista.Persona
 - Partecipa.Escursione → Escursione.id

5 Implementazione in PostgreSQL e definizione delle Query

Il file `escursioni.sql` contiene il codice SQL necessario per la creazione e il popolamento delle tabelle del database. Questo file include inoltre una serie di Query per l'estrazione dei dati e un indice creato specificamente per migliorare le prestazioni di una di queste interrogazioni.

5.1 Definizione delle Query

Di seguito sono descritte le Query con i relativi output generati

Query 1

```

1  SELECT g.cf, p.nome, p.cognome, COUNT(e.id_escursione) AS numero_escursioni
2  FROM GUIDA g
3  JOIN PERSONA p ON g.cf = p.cf
4  LEFT JOIN ESCURSIONE e ON g.cf = e.cf_guida
5  GROUP BY g.cf, p.nome, p.cognome
6  ORDER BY numero_escursioni DESC;
```

Query 2

	cf character (16)	nome character varying (100)	cognome character varying (100)	numero_escursioni bigint
1	JRTNTN97D14L21...	Giovanni	Lombardi	7
2	SMRGIO89D49L219X	Samartitano	Giorgio	6
3	ARGCHR83S45H50...	Chiara	Argento	6
4	QRLTNL94T62H501U	Marcella	Tufano	4
5	CLTANG79P68M31...	Angelo	Colato	4
6	SFTNTN81M48M3...	Sofia	Morandi	3
7	DRGCCM69S57H50...	Maria	Matteucci	3
8	RLTCSM91D48H50...	Margherita	Malavolta	3
9	DSSNMM02R27D2...	Fabio	Martino	3
10	NGFLST71L11M31...	Simone	Mocchetti	3
11	PFGNTL79C31H50...	Stefano	Cavalli	3
12	GQNTNT87A16L21...	Luca	De Santis	3
13	FMNTNT69M44D2...	Simona	Rovelli	3
14	MRLTLL08R48H501X	Martina	Di Luigi	3
15	GRNTNR99T15L21...	Giovanni	Gallo	2

Figura 3: Risultato o esecuzione della Query 1

```

1  DROP VIEW IF EXISTS V_NUMERO_TAPPE;
2
3  CREATE VIEW V_NUMERO_TAPPE AS
4  SELECT s.codice, s.nome, COUNT(a.id_tappa) AS numero_tappe
5  FROM SENTIERO s
6  JOIN attraversa a ON s.codice = a.codice_sentiero
7  GROUP BY s.codice, s.nome;
8
9  SELECT codice, nome, numero_tappe
10 FROM V_NUMERO_TAPPE
11 WHERE numero_tappe = (
12     SELECT MAX(numero_tappe)
13     FROM V_NUMERO_TAPPE
14 );

```

	codice integer	nome character varying (100)	numero_tappe bigint
1	109	Anello Cima Carega da Rifugio Revo...	4

Figura 4: Risultato o esecuzione della Query 2

Query 3

```

1  SELECT e.id_escursione, e.data_programmata, s.nome AS sentiero, p.nome, p.
   cognome, e.costo
2  FROM ESCURSIONE e
3  JOIN SENTIERO s ON e.codice_sentiero = s.codice
4  JOIN GUIDA g ON e.cf_guida = g.cf
5  JOIN PERSONA p ON g.cf = p.cf
6  WHERE e.data_programmata BETWEEN '2026-06-13' AND '2026-12-24'
7  ORDER BY e.data_programmata;

```

	id_escursione integer	data_programm date	sentiero character varying (100)	nome character varyin	cognome character var	costo numeric (8,2)
1	1992948	2026-06-13	Alta Via n. 2 delle Dolomiti - Tappa al Rifugio Dal Piaz	Luca	Rossi	150.00
2	1992944	2026-06-13	Lago della Stua e Rifugio Bruno Boz - giro ad anello	Giovanni	Lombardi	150.00
3	1992957	2026-06-13	Strada delle 52 gallerie del Pasubio	Stefano	Cavalli	150.00
4	1992959	2026-06-14	Anello di Busa Novegno	Giovanni	Lombardi	85.00
5	1992974	2026-06-14	Sentiero C.A.I. n° 511 (Rifugio VII Alpini - Monte Pelf)	Samaritano	Giorgio	150.00
6	1992969	2026-06-18	Sentiero CAI 811	Simone	Mocchetti	150.00
7	1992929	2026-06-18	Anello Cima Carega da Rifugio Revolto	Angelo	Colato	150.00
8	1992926	2026-06-21	Sentiero C.A.I. n° 505 (Val Vescovà - Rifugio Bianchet)	Simona	Lombardi	150.00
9	1992940	2026-06-21	Sentiero C.A.I. n° 10 (Cascate di Fanes)	Luca	De Santis	85.00
10	1992964	2026-06-21	Alta Via n. 2 delle Dolomiti - Tappa al Rifugio Dal Piaz	Maria	Matteucci	150.00
11	1992960	2026-06-22	Sentiero C.A.I. n° 157 (Boale dei Fondi)	Giovanni	Lombardi	150.00
12	1992949	2026-06-23	Salita al Forte Enna	Margherita	Malavolta	90.00
13	1992924	2026-06-23	Anello del Pasubio: Strada degli Eroi	Chiara	Argento	150.00
14	1992905	2026-06-23	Sentiero C.A.I. n° 353 (Val Montanaia)	Maria	Matteucci	150.00
15	1992951	2026-06-27	Sentiero C.A.I. n° 104 (Locatelli - Pian di Cengia)	Margherita	Malavolta	80.00

Figura 5: Risultato o esecuzione della Query 3

Query 4

```

1  SELECT e.cf, p.nome, p.cognome, COUNT(pa.id_escursione) AS numero_escursioni
2  FROM ESCURSIONISTA e
3  JOIN PERSONA p ON e.cf = p.cf
4  JOIN partecipa pa ON e.cf = pa.cf_escursionista
5  GROUP BY e.cf, p.nome, p.cognome
6  HAVING COUNT(pa.id_escursione) >= 6
7  ORDER BY numero_escursioni DESC;

```

Query 5

```

1  SELECT g.cf, p.nome, p.cognome, ROUND(AVG(e.costo),2) AS costo_medio
2  FROM GUIDA g
3  JOIN PERSONA p ON g.cf = p.cf
4  JOIN ESCURSIONE e ON g.cf = e.cf_guida
5  GROUP BY g.cf, p.nome, p.cognome
6  ORDER BY costo_medio DESC;

```

	cf character (16)	nome character varyin	cognome character var	numero_escursioni bigint
1	TMRBMT79R52H501V	Lucia	Mazzola	9
2	QTRNMT87D17D205X	Giovanni	Ferro	8
3	ARGCHR83S45H501Z	Chiara	Argento	8
4	PRGRNS82H28D205X	Giorgio	Tarantino	8
5	JGNPRT87T30M312X	Giovanni	Galli	7
6	QNLRNT92P46D205X	Francesca	Barbieri	7
7	DHLMSC81M50H501Y	Francesca	Masi	7
8	MNTCLS72E43M312Y	Miriam	Montagna	7
9	GFNTNT90B28D205X	Luca	Fioretti	7
10	QNGVRE71T51D205V	Francesca	Gualtieri	7
11	RJSFRN96T17M312V	Francesco	Giordano	6
12	TFNSTT80A64M312S	Simona	Martino	6
13	DGFQNT88B29D205X	Luca	Falcone	6
14	LVRNTT64R10M312X	Riccardo	Sannino	6
15	PNRRNS88P28D205X	Riccardo	Nuzzi	6

Figura 6: Risultato o esecuzione della Query 4

	cf character (16)	nome character varyin	cognome character var	costo_medio numeric
1	QMGNMR82B28H501T	Riccardo	Marchesi	150.00
2	QBRLMT00A52D205V	Angela	Di Mauro	150.00
3	QNTNTT76S60M312S	Simona	Morandi	150.00
4	SPSLRG82M59D205W	Francesca	Pietro	150.00
5	CPTVRS85E04D205X	Enrico	Zanetti	150.00
6	LTLNNT97R45D205W	Francesca	Rossi	150.00
7	LMNTNT85R28L219T	Luca	Rossi	150.00
8	GMSNTR75E54M312X	Simona	Lombardi	150.00
9	GRNTNR99T15L219T	Giovanni	Gallo	147.50
10	LVFTRN00L14M312S	Alessandra	Tartaglia	135.00
11	ARGCHR83S45H501Z	Chiara	Argento	135.00
12	PFGNTL79C31H501V	Stefano	Cavalli	126.67
13	SMRGIO89D49L219X	Samartitano	Giorgio	124.17
14	DRGCCM69S57H501Y	Maria	Matteucci	123.33
15	JRTNTN97D14L219V	Giovanni	Lombardi	120.71

Figura 7: Risultato o esecuzione della Query 5

5.2 Creazione degli indici

Per ottimizzare la Query è stato scelto un indice di tipo **B-tree** sull'attributo **data_programmata** della tabella **ESCURSIONE**. Questa scelta è motivata dal fatto che la query effettua una ricerca su un intervallo di date mediante l'operatore **BETWEEN** e restituisce i risultati ordinati cronologicamente tramite **ORDER BY**. La struttura B-tree mantiene i valori dell'indice in ordine, consentendo di individuare rapidamente l'inizio dell'intervallo richiesto e di scorrere solo le tuple comprese tra le due date, evitando una scansione completa della tabella. Inoltre, poiché i dati sono già ordinati nell'indice, anche l'operazione di ordinamento risulta più efficiente. Un indice di tipo **Hash** non sarebbe invece adatto, poiché è progettato esclusivamente per ricerche basate sull'uguaglianza (=) e non supporta in modo efficiente ricerche per intervallo o operazioni di ordinamento. Per questi motivi, il B-tree rappresenta la scelta più appropriata per questa query.